

ESP32 Network Toolkit

Arbeitsbericht und Überblick über Aufbau, Funktionen und Grenzen

Das ESP32 Network Toolkit ist ein Lernprojekt zu Netzwerken, serieller Kommunikation und Log-Auswertung. Ein ESP32 führt die WLAN- und TCP-Prüfungen aus; Bash, PowerShell und Python steuern das Gerät und verarbeiten die Ergebnisse.

Einsatzbereich: eigene oder ausdrücklich autorisierte lokale Netze. Ein manuelles Ziel ist auf private IPv4-Adressen im verbundenen WLAN-Subnetz begrenzt.

Komponenten

Bereich	Aufgabe
ESP32-Firmware	WLAN-Übersicht, lokale Geräteerkennung, Portprüfungen, mDNS und Log-Aufzeichnung.
Bash	Menü, serielle Steuerung sowie Profilfilter und Zusammenfassungen der Logs.
PowerShell	Verbindet den Windows-PC über den seriellen Port mit dem ESP32.
Python	Terminaloberfläche, Log-Normalisierung, SQLite-Verlauf und Markdown-Berichte.

Projektstruktur

firmware/ enthält den Arduino-Sketch und eine Beispieldatei für lokale Zugangsdaten. host/bash/ und host/powershell/ enthalten die PC-Steuerung. analysis/python_ui/ enthält Auswertung und UI; profiles/ enthält die Portprofile.

Ablauf und Funktionen

Nach dem Start verbindet sich der ESP32 mit dem eingetragenen WLAN. Die Steuerung am PC sendet eine Menüauswahl über den seriellen Port und speichert die zurückgegebenen Zeilen in einem Log.

- WLAN-Übersicht: zeigt erreichbare WLANs und deren Signal- und Verschlüsselungsangaben.
- LAN-Erkennung: prüft nutzbare Adressen des verbundenen Subnetzes auf ausgewählte TCP-Ports und fragt mDNS-Namen ab.
- Gateway-Prüfung: prüft ausgewählte TCP-Ports des WLAN-Gateways.
- Manuelles Ziel: prüft eine eingegebene private IPv4-Adresse im verbundenen Subnetz; Geräteadresse, Netzwerkadresse und Broadcast-Adresse sind ausgeschlossen.
- Profile: ordnet offene Ports heuristisch Router-, PC- oder Serverprofilen zu. Das Ergebnis ist eine Schätzung, keine sichere Geräteidentifikation.

Logs und Berichte

Bash kann Logs zusammenfassen, nach Profilen filtern und ein Profil aus den gefundenen Ports schätzen. Die Python-Werkzeuge normalisieren Logs, speichern Scan-Metadaten in `data/scans.db` und erzeugen Markdown-Berichte unter `analysis/reports/`. Die Portliste beschreibt nur, dass eine TCP-Verbindung angenommen wurde; sie bestätigt nicht die Identität oder Sicherheit des dahinterliegenden Dienstes.

Start unter Windows

Schritt	Aktion
1	Python 3.10+, Git Bash, PowerShell und Arduino-ESP32-Core bereitstellen.
2	Pakete installieren: <code>python -m pip install -r requirements.txt</code> .
3	<code>firmware/secrets.example.h</code> nach <code>firmware/secrets.h</code> kopieren und die lokalen WLAN-Zugangsdaten eintragen.
4	Ordner <code>firmware/</code> in Arduino IDE öffnen, Firmware hochladen und den seriellen Monitor schließen.
5	<code>python analysis/python_ui/app.py</code> oder <code>host/bash/main.sh</code> starten.

Sicherheit und technische Grenzen

- Nur Geräte und Netze prüfen, für die eine ausdrückliche Berechtigung besteht.
- Manuelle Ziele müssen private IPv4-Adressen im aktuell verbundenen WLAN-Subnetz sein.
- Die automatische LAN-Erkennung wird bei mehr als 254 nutzbaren Hostadressen abgebrochen, damit keine großen Netze ungeplant abgearbeitet werden.
- Die Zugangsdaten gehören in die lokale, von Git ignorierte Datei `firmware/secrets.h`; nur die Beispieldatei wird versioniert.
- Die Firmware formatiert SPIFFS nicht automatisch, wenn das Dateisystem nicht eingehängt werden kann. So werden gespeicherte Logs bei einem Mount-Fehler nicht automatisch gelöscht.

Was die Ergebnisse aussagen

Ein offener TCP-Port bedeutet, dass ein Verbindungsversuch angenommen wurde. Das sagt allein weder aus, welcher Dienst antwortet, noch ob er korrekt verschlüsselt oder verwundbar ist. Die Router-, PC- und Serverprofile sind einfache Heuristiken aus ausgewählten Ports und sollten manuell geprüft werden.

Lernziele

Das Projekt verbindet Hardware, WLAN, TCP/IP, serielle Kommunikation und mehrere Programmiersprachen zu einem nachvollziehbaren Ablauf. Beim Arbeiten damit lassen sich Netzwerk Begriffe, Log-Verarbeitung, SQLite und einfache Automatisierung praktisch nachvollziehen.

Weiterentwicklung

Sinnvolle nächste Schritte sind ein vollständiger Arduino-Build mit dem konkreten Boardprofil, ein Test am eigenen ESP32 sowie Beispiel-Logs für die Bash- und Python-Auswertung. Diese Prüfungen sind von der hier beschriebenen statischen Code- und Dokumentationsprüfung getrennt.